ERMS

Expense Reimbursement Management System

Judges' FAQ

KSCAA Tech RRC 2026 - Group 8

Prepared July 2026

Architecture & Technology Choices

Q1. Why React + Node/Express + PostgreSQL instead of a no-code platform or a different stack?

Expense reimbursement is fundamentally a relational, transactional problem: a claim has line items, moves through a strict sequence of approval steps, and must reconcile exactly against payments. PostgreSQL's relational integrity and transactions are a natural fit for that - a document database would need to reinvent joins and referential integrity by hand. Node/Express keeps the whole stack in one language (TypeScript), which matters for a small team splitting the system into five pieces in parallel. Prisma sits on top of Postgres as the single schema source of truth so the data model can't silently drift between the five workstreams.

Q2. Why split the system into exactly these five pieces (Employee, Manager, Accounts, Admin, Reports)?

That split mirrors the actual roles in the problem statement, not an arbitrary technical division. Each piece owns a real, separable part of the claim lifecycle: submission, first-level approval, financial verification/payment, master data & access control, and reporting. That means each of the five people has a genuinely different design problem to solve - not five people editing the same screen.

Q3. How is the database schema designed to support all of this in one consistent model?

One Prisma schema is the single source of truth (`packages/db/prisma/schema.prisma`). Every module reads and writes the same `Claim`/`ApprovalStep`/`AuditLog` tables rather than forking its own copy of the data model, which is what keeps five parallel workstreams from drifting apart.

Security & Internal Controls

Q4. How exactly do you prevent someone from approving their own claim?

Every approval action checks the specific manager-employee relationship in the database (`claim.employee.managerId === reviewer.id`) - not department membership, not role alone. During our own testing we found and closed a real gap where this check existed on the approval-queue listing but not on the actual decision endpoint, meaning any manager could approve any claim in their department. It's now enforced identically on both. The one deliberate exception is the CEO, who has no manager above them by definition and is the only role permitted to approve their own claim - there is structurally no one else who could.

Q5. Is the audit trail actually tamper-proof, or just a log table?

It's append-only by convention and by API surface: there is no update or delete route for `AuditLog` anywhere in the system, and every mutation - claim submission, every approval decision, payment, login, password change - writes exactly one row through a single shared helper (`recordAuditLog`), so no code path can mutate data without also leaving a trail. Admin can view, filter, and export it, never edit it.

Q6. How does duplicate/fraud detection actually work?

Deterministically today: a claim is flagged if the same employee submits the same amount within a 7-day window of another claim, or if two claims share a bill attachment with an identical file hash. This check runs automatically the moment Accounts attempts to verify a claim - a flagged claim cannot be verified or paid, full stop, not just shown a warning. The AI roadmap item (ML-based fraud scoring) is designed to augment this baseline, not replace it, since a deterministic check is auditable in a way a model score alone isn't.

Q7. How do you handle password security and session timeout?

Passwords are bcrypt-hashed (never stored or logged in plaintext) and must meet a strength policy (8+ characters, upper/lowercase, number, special character) enforced identically whether an employee changes their own password, an admin creates an account, or a reset link is used. First login and any admin-triggered reset force a password change before anything else in the system is reachable. Session idle timeout is tracked against a database-backed "last active" timestamp updated on every request - deliberately not baked into the JWT itself, since a signed token is immutable once issued and can't reflect real subsequent activity.

Q8. What's the most serious bug you found while testing this yourselves?

A path traversal vulnerability in bill uploads: the claim ID from the URL was used directly as a folder name before the system had confirmed the claim was real or belonged to the uploader, so a crafted ID containing `../` sequences could write a file outside the application entirely. We caught it by deliberately attacking our own upload endpoint, fixed it by validating ownership before any file-system operation runs, and added a belt-and-suspenders check that refuses to create any directory outside the intended uploads folder even if that ownership check were ever bypassed.

Business Logic & Workflow

Q9. What happens to a claim that's "returned" by a manager?

It goes back to the employee with the manager's remarks shown directly on an edit screen - not just a blind resubmit button. The employee can change line items and swap attachments in response to those remarks before resubmitting; the same mandatory-attachment rule applies again either way, so a resubmission can't skip that control. Resubmission re-enters the same manager's queue, and the manager is notified.

Q10. What happens if a manager is deactivated while they still have direct reports?

The system refuses the deactivation outright, with a clear message to reassign those reports first. Since approval is strictly scoped to the assigned manager, deactivating one without reassigning their reports would leave those employees' claims permanently unreachable - no one else could approve them, not even an admin override.

Q11. Can two people accidentally approve or pay the same claim twice?

No - every status-changing action (approve/reject/return, verify/reject, and payment) uses an atomic conditional database update that only succeeds if the claim is still in the expected state at that exact instant. A second, near-simultaneous attempt on the same claim is rejected with a clean "already processed" error instead of silently double-processing it.

Q12. Why enforce a per-category spending limit instead of just a total claim limit?

Real expense policies are usually per-expense-type (a Rs. 2,000 meal limit vs a Rs. 25,000 travel limit), so the system validates each line item against its own category's limit at submission time, not just an aggregate total - a claim mixing a compliant travel expense with a policy-violating meal expense is caught immediately rather than averaging out.

Scalability & What's Next

Q13. The Approval Matrix screen exists but multi-level approval isn't live - why?

The data model (department/amount-based rules with an approver sequence) is fully built and admin-configurable, but wiring it into live routing needs a new claim-status state to represent "awaiting second-level approval," and we deliberately chose not to bolt that onto a claim-approval path we'd just finished hardening for correctness under concurrent access. It's a real feature, not a stub, but it needs its own careful pass rather than a rushed addition.

Q14. How would this scale to a real organization with hundreds or thousands of employees?

The data model already supports it structurally - departments, managers, and approval steps are all relational and indexed for the query patterns each screen actually uses (a manager's queue is a direct index lookup on `managerId`, not a table scan). What would need attention at real scale: object storage instead of local disk for bill attachments, a proper email service for password resets and notifications, and pagination on list screens that currently return full result sets.

Q15. What's the AI roadmap, concretely?

Two items, both additive rather than load-bearing: OCR-based extraction to pre-fill claim line items from a photographed receipt, and an ML-based fraud/anomaly score that augments the deterministic duplicate check rather than replacing it - the deterministic check stays as the auditable compliance baseline regardless of what a model says.

Development Process

Q16. How did you verify this actually works, rather than just compiles?

Every feature in this document was exercised end-to-end against a running instance with real HTTP requests - login, submit, approve, verify, pay, reject, resubmit, reset a password, view the audit trail - not just unit-level assertions. Several real bugs (the path traversal issue, an unenforced duplicate-flag check, a session-timeout that never actually reset) were only caught this way, by deliberately trying to break the

system the way a real user or attacker would.

Q17. What would you build next if given another week?

In priority order: live multi-level approval routing from the Approval Matrix, Excel/PDF report export alongside the existing CSV export, MFA enrollment (the schema already has the fields for it), and emailing the in-app notifications that already exist rather than requiring a user to check the Notifications screen.

Q18. Can a manager approve only part of a claimed amount?

Yes - a manager isn't limited to a binary approve/reject/return; on Approve they can enter an amount up to (but not exceeding) the claimed total, e.g. approving the flight but not an out-of-policy hotel line item within the same claim. That approved amount, not the original total, is what Accounts verifies and ultimately pays - the claim's own `totalAmount` stays as the permanent record of what was originally claimed, while `approvedAmount` (on both the claim and its `ApprovalStep` audit row) records exactly what was authorized and by whom. The employee's notification and Claim History both show the split clearly when the two amounts differ.